UNCORRELATED ORBITAL TRACK PROCESSING

Preliminary Design Document

Kyle Galang, Aurela Broqi, Ruben Dennis, Aaron Nogues, Ezra Stone

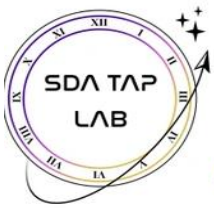
COP4934: Computer Science Senior Design I

Dr. Richard Leinecker

University of Central Florida

Orlando, Florida

October 31, 2025



AFRL

Table of Contents:

Housekeeping: 1.1 Definitions

Nomenclature	
Term	Meaning
UCTP	Uncorrelated Track Processing
LLM(s)	Large Language Model(s)
HPC	High-Performance Computing Clusters
SDA TAP Lab	Space Domain Awareness Tools, Applications, & Processing Lab
AFRL	Airforce Research Laboratory
DARPA	Defense Advanced Research Projects Agency
TLE	Two Line Element
GUI	Graphical User Interface
UDL	Unified Data Library: the source of state vector and observation data used in UCT
TLE	Two-line element set, also known as elset
HMDH	Hierarchical Multi-Dimensional Histograms

Housekeeping: 1.2 Document Breakdown

This document is structured in a way of how to solve sponsor and senior design requirements. The chapters given are the separate topics that all need to be researched in order to meet the requirements to solve how OpenEvolve can help benchmark UCT. All the chapters are all interlinked with one another and have been separated by different group members to explain what we are researching and how we are applying it back to sponsor and senior design requirements. There are many new topics that needed to be researched in order to understand how to integrate OpenEvolve's AI architecture into an already existing black-box model created into uncorrelated track processing by SDA TAP lab.

The architecture of how to solve this problem and meet requirements goes as follows:

- Uncorrelated Track Processing (Already created and need to extract JSON data from satellites)
- OpenEvolve AI architecture to help benchmark that output data to see how well UCT performs
- Symbolic Regression (OpenEvolve's prime example on how they benchmark with already known solutions of 239 scientific domain problems)
- Graphical User Interface (To track OpenEvolve's iteration in real time)
- LLM (Trained from the example problems in order to help write out the initial and evaluator functions needed in the OpenEvolve architecture)

These five different topics have been separated by each group member to explain in more depth the research and development being done to meet both senior design software design and development requirement and sponsor requirements of pure research documentation and poster.

Project Concept of Operations: Design Plan 2.1

The concept of operations for this senior design project is in conjunction with the AFRL, SDA TAP Lab, MITRE, and another student group. This project focuses on OpenEvolve which is an open-source adaptation of Google DeepMind's AlphaEvolve agent that automates algorithm discovery and code optimization through large language models (LLM). The overarching goal is to research OpenEvolve architecture and learn how the operations work in order to implement it to UCTP. At the moment UCTP is able to make observations or tracks of observations that do not correlate to any known objects. But the problem is solutions to UCT processing do not have a known solution to compare to. Which is why OpenEvolve is essential, which will help benchmark datasets with known solutions and performance metrics to evaluate UCTP performance. How we will perform this project is to understand how OpenEvolve architecture works through their symbolic regression example. The symbolic regression problem benchmarks two-hundred and thirty-nine scientific domain problems with known solutions which will help evaluate UCTP performance. We will then create smaller problem sets that correlate to UCTP to test OpenEvolve's features using Google AI Studio API to query into OpenAI LLM. Since we will be working with larger datasets more common to UCTP outputs. If we find that the AI architecture of OpenEvolve fits the criteria of benchmarking UCTP. We will then move our problem sets to MITRE high-performance computing clusters to test OpenEvolve.

The second part of this project is to meet senior design requirements; our team is split into two. One is pure research and development using OpenEvolve for sponsoring requirements and goals as explained in the previous paragraph. While the other is senior design requirements through software design and development. The software design and development team will be creating a dashboard that connects locally to OpenEvolve's repository which will showcase real-time efficiency scores. It will also have an LLM input and output feature which will be trained from OpenEvolve's example problems to generate initial and evaluator programs.

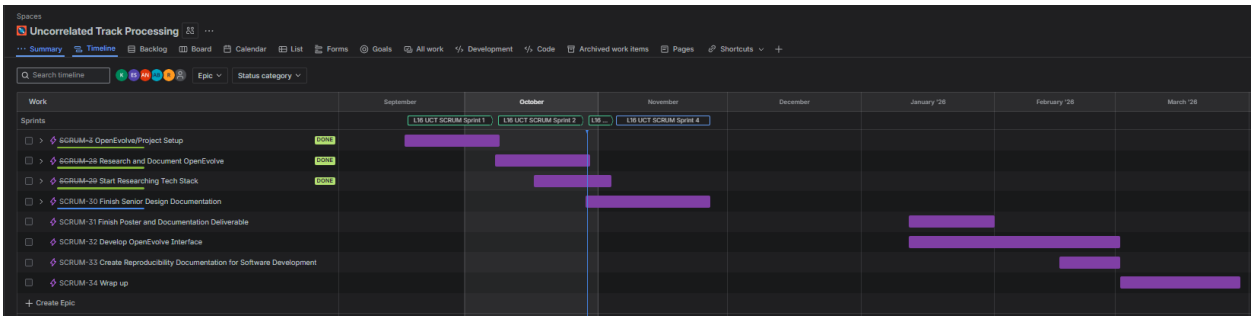
Project Concept of Operations: Minimum Viable Product 2.2

Our minimum viable product is ever so slightly changing every week as new research topics and taskings are allocated among members of the group alongside sponsors. But consistent goals from the sponsor consist of 4 minimum viable products to satisfy the sponsors. The first one being a full research document that determines the viability of OpenEvolve's AI architecture as a benchmarking tool for uncorrelated track processing. Second being a poster that summarizes our findings in OpenEvolve and showcasing our data points in charts and graphs from their specified colored schemes. These first two minimum viable products are what is expected from AirForce Research Laboratory and sponsors. But in order to fulfill the other half of the class for senior design, we need to have requirements that are senior design worth which includes software design and development. Two additional minimum viable products that have been negotiated and accepted with our sponsors, professors, and teaching assistants are creating a GUI that tracks the iterations of OpenEvolve in real time to see how efficient the algorithms are per iteration. While the last one being an LLM that code generates two specific types of files for OpenEvolve to work. Which is the initial program which states the algorithm or equation as a Python function and the other one being an evaluator Python file which states what type of metrics are being used to score efficiency per iteration. The software and development side of the minimum viable product are also all documented with how to use the interface dashboard and how it was developed in case our sponsors want to expand or replicate our development.

Project Concept of Operations: Timeline and Milestones 2.3

Sprint map timeline/milestones from now to the end of March:

Figure 2.31:



Current progress and milestones:

Figure 2.32:

Note:

Since our group is split between two teams, one timeline we had to account for is the sponsor tasks. Slowly every sponsor meeting we are getting closer to the true intentions of OpenEvolve. It is hard to distinguish what is done and what is not done in a research-based project. Especially since our sponsors told us that the HPC is still being set up and that we might not be able to fully implement OpenEvolve into UCT before senior design 2 ends. The timeline with the access of the HPC and running problems in it is in March according to the sponsors. Our team is trying our best to keep going in regard to morality through short tasks due to the long timeline of this project.

What our team aims to do is be able to have an environment in the HPC through Open OnDemand which allows us to have remote web access to the supercomputer by December. Alongside majority of the research and documentation done. As seen in the figures 2.31 and 2.32, we aim to focus on GUI/LLM development and poster/documentation in the first month of Senior Design 2. But this is a speculation since new tasking and roadblocks ahead can hinder those timelines.

Uncorrelated Track (UCT) Processing: Overview & Data Breakdown 3.1

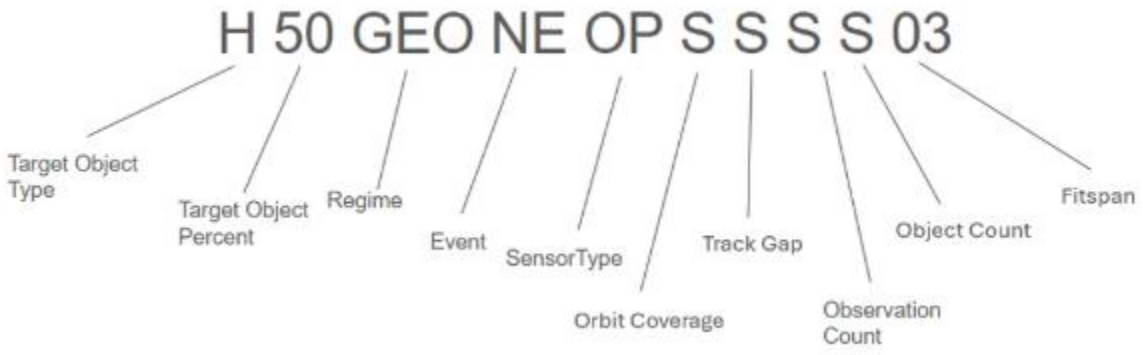
What is UCT Processing?

Uncorrelated Track (UCT) processing is the analysis of orbital data to correlate orbits to objects. All objects in orbit follow the laws of physics and thus their heading and origin are both able to be calculated after some observations in motion. These and many more observations will be provided by the sponsor, via generated data sets or the Unified Data Library (UDL). For the sake of reference, this will be referred to as “given data” or “given data sets” instead of specifying which type of data it is, generated or not.

Given Data Sets

These given data sets will be given according to the desired characteristics input by the end user. The characteristics of the dataset will be encoded in a 16-character string shown below:

Figure 3.10:



Target Object Types

The target object types are a simple overall classification of the type of object to be given and/or later simulated. The relationship between the letter of classification and characteristics of this classification is defined by the graph:

High Area to Mass Ratio (HAMR)	Close Objects	Close Apparent Objects	Unspecified	Calibration
> 1 m^2/kg	distance<X km velocity<X m/s	Angles within: X °	Datasets unconstrained by object types	Well-tracked objects only
H	C	A	U	N

Target Object Percent

The target object percentage is the percentage of the target object to be included in the given dataset. Any value from 100 to 01 or UN (unspecified) is valid. When UN is chosen it leaves the makeup of the data set to the choice of the algorithm, populating the data set with a natural distribution of objects. The reason it is directly correlated with the object type is because objects of type Unspecified must have UN as their target object percent.

Orbital Regime

The orbital regime is the measurement of the semi-major axis a in kilometers. There are 4 regimes Low Earth Orbit (LEO), Middle Earth Orbit (MEO), Geosynchronous Orbit (GEO) and Highly Elliptical Orbit (HEO) and the orbital regime of the object can take up any number of combinations of these 4 regimes. These combinations are LMO, LGO, LHO, MGO, MHO, GHO, LMG, LGH, MGH, and ALL. Combinations of 2 are the first letters of the unique word from the 4 regimes (L, M, G, H) followed by an O to stand for “orbit” and combinations of 3 are the first letters paired together. All letters are sorted by the order of their regime, L being first, H being last and so on. The range of a by its regime are defined by the chart:

Manuever Between OBS	Breakup	Long Duration/low thrust	No Events
MB	BU	LL	NE

Sensor Type

The type of sensor that would pick up the object in orbit (when searched or generated).

Optical	Radar	Radio Frequency (RF)	Fusion of all sensor types	Optical/RF	Optical/Radar	Radar/RF
OP	RA	RF	FU	OR	RO	RR

Orbit Coverage

The orbital coverage is the amount of the object's path covered by the data, based on its orbital regime there are accepted minimum values for identification based on how covered the object in orbit is.

The relationship between the orbital regime and the minimum (not Low coverage) acceptable orbital coverage to identify the object in orbit is shown by the chart.

LEO (Low <0.0213% covered)
MEO (Low <0.0449% covered)
GEO (Low <41.656% covered)

The letters that represent orbital coverage are A, S, N, and the values are defined by the chart below:

>90% of objects have low orbital coverage	40-60% of objects have low orbital coverage	<10% of objects have low orbital coverage

A	S	N

Track Gap

The track gap is the longest duration of time between observations of an object, the shorter the better. According to our sponsors, a 'long' track gap is defined as a duration of two orbital periods of the object. When preparing the example data, Orbital Regime and Fitspan take precedence over track gap which means the track gap; when given, is more likely to be considered 'long'. This infers that track gap as a metric for identifying objects is less important, which is possibly why the generated data is less 'reliable' than the data directly from the UDL. The convention also uses the same letters that Orbital Coverage uses but it is important to note that the value of representation is different than Orbital Coverage:

>90% of objects have a long track gap	40-60% of objects have a long track gap	<10% of objects have a long track gap
A	S	N

Observation Count

The observation count is the total number of unique observations per object per 3-day-timespan. A 'low' observation count is less than 50, standard' observation count is between 50-150 and objects with greater than 150 observations will not be used or will be down sampled to a 'standard' observation count. This uses the same letters that Orbital Coverage uses but it is important to note that the value of representation is different than Orbital Coverage:

>90% of objects have low observation count	40-60% of objects have low observation count	<10% of objects have low observation count
A	S	N

Object Count

The object count is the total number of objects to be included in the object data set give or take 2 objects, it is identified by three letters H, S, L. The values of the letters are defined by the chart below:

High number of objects: 80 +/- 2	Standard Number of Objects: 40 +/- 2	Low number of objects: 10 +/- 2
H	S	L

Fit span

The fitspan is the duration of the dataset spans in days from 01 to 14.

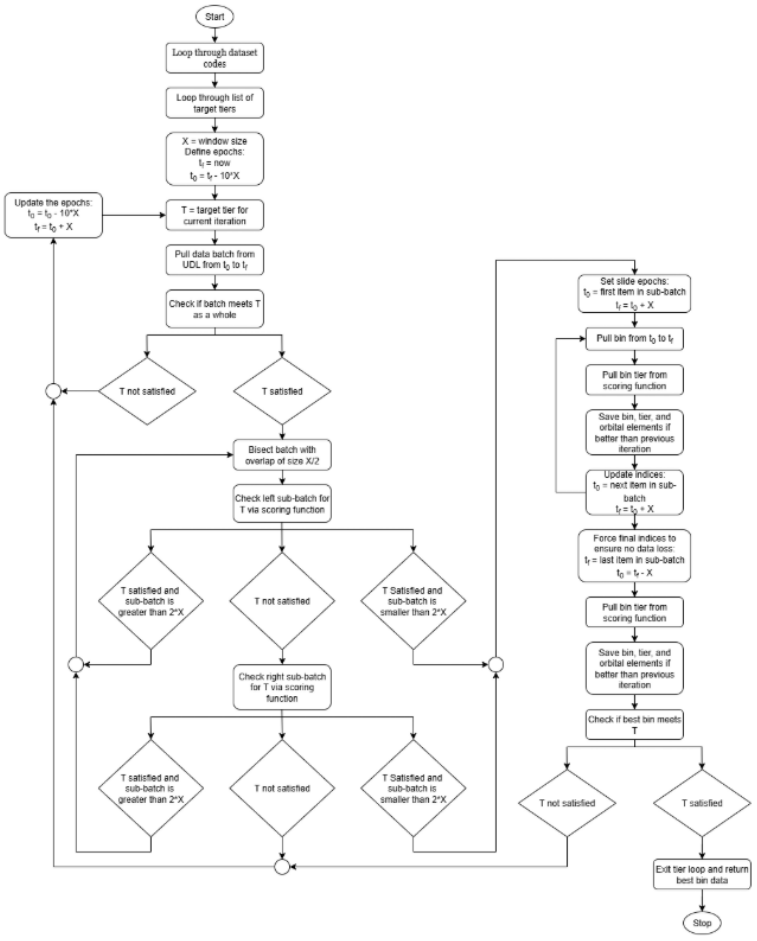
Actively Assembling the Dataset. Fit metrics determines the unit-sphere projected great circle residuals between the actual observations and the propagation observations. The function is split into two categories:

- Comparison of the reference (truth) observations with the associated orbits (how accurate a correlated state is)
- Comparison of the reference (truth) observations with the UCTP output (how precise a correlated state is)

The character string makes up the queries that. The steps to then use these parameters for the given datasets are as follows: (on the following page)

The first step in creating a dataset is to identify the window as stated previously. The most optimal time is when there is no simulation or down sampling required. Which is shown on the following chart.

Figure 3.11:



The logic chart isn't as fully in depth but follows along these rules for evaluating T to return the bin data:

Time Window Selection

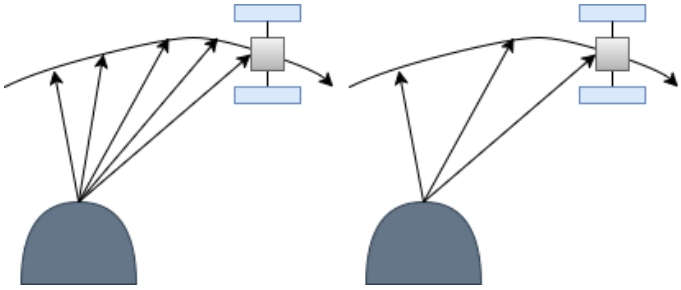
The first step to fully preparing and giving the data is identifying an ideal time window based off of the fitspan and other variables present in the string. The resulting quality of time window selection is separated into tiers according to this chart:

Tier 1	Tier 2	Tier 3	Tier 4	Tier 5
(optimal) No downsampling No simulation	No simulation	Simulation of objects already present	Simulation of new objects	(worst) Criteria is impossible

Uncorrelated Track (UCT) Processing: Data Manipulation 3.2

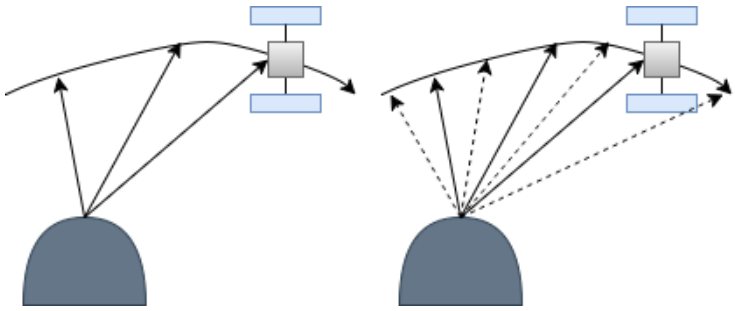
Downsampling

If the observational data is too high of a quality (too many data points than the string requested), the data must be simplified by removing in order of importance. The order of importance outlined by our sponsors is Orbit Coverage, Track Gap and Number of Observations. They remove readings from the satellites with the lowest orbit coverage, then lowest track gap and then lowest number of observations. This assumes that satellites further in orbit are more accurate.



Simulation

Inside the chart which defines the tiers, tier 3 and 4 have both to do with simulation. The key difference between them is the Tier 3 simulation uses data from a real object that exists in the data set, i.e. moving an object in orbit to a different point in space and Tier 4 creates a new object in the data set because a sufficient number of reference objects do not exist in the data set.



True Negatives

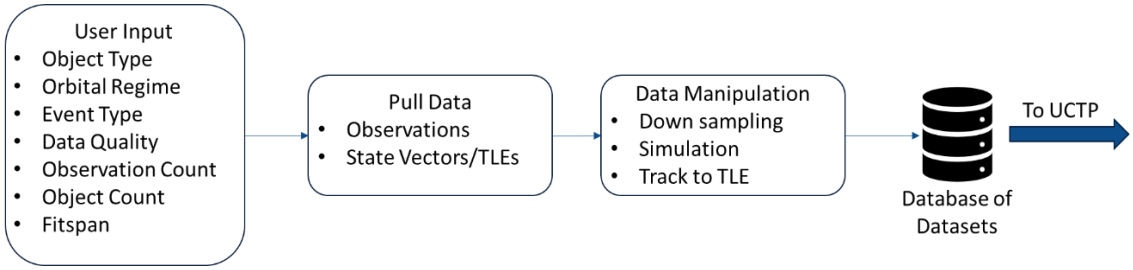
Datasets given will include some percentage of true negatives or observations that do not correlate with any of the reference objects, which implies our evaluator script must be able to identify objects as unknown or as true negatives. True negative data points are added to the given data after all other data manipulation is complete.

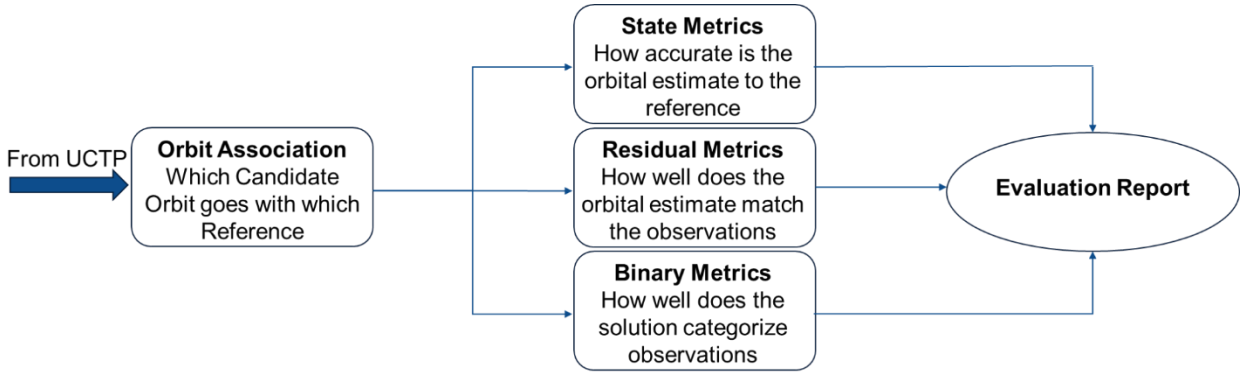
Decorrelation

After all the data manipulation, the data set becomes decorrelated, removing each corresponding object's ID and observations will be dissociated from the reference orbits. Decorrelated objects do keep their observational passes (the number that identifies which observatory found it) now that the data has been decorrelated, the given data and an "answer key" are now produced and ready for our team to use.

Dataset Generation

With all the variable names identified, the dataset generation workflow will give an output regarding the dataset code in JSON format, which then our group will gather in order to benchmark UCT against OpenEvolve's evaluation.


User Input
Object Type
Orbital Regime
Event Type
Data Quality
Observation Count
Object Count
Fitspan
Pull Data
Observations
State Vectors/TLEs
Data Manipulation
Down sampling
Simulation
Track to TLE
To UCTP
Database of Datasets


From UCTP
Orbit Association
Which Candidate Orbit goes with which Reference
State Metrics
How accurate is the orbital estimate to the reference
Residual Metrics
How well does the orbital estimate match the observations
Binary Metrics
How well does the solution categorize observations
Evaluation Report

OpenEvolve Framework: Overview & Data Breakdown 4.1

3.1 Introduction

OpenEvolve is an open-source framework that is designed to automate algorithm discovery and code optimization. Instead of relying on traditional and manual tuning methodology, OpenEvolve leverages large language models (LLMs) and evolutionary computation to autonomously generate, modify, and evaluate code. Each iteration of the system mimics biological evolution. Programs will go through small and structured edits or "mutate", then they will be evaluated according to defined metrics. The most successful mutations are preserved for future generations.

For this project, OpenEvolve is being researched as a potential tool for evolving and optimizing algorithms used in Uncorrelated Track Processing (UCTP). UCTP systems rely on computationally intensive models, such as linear and symbolic regressions, to estimate object trajectories and correlations in noisy or incomplete sensor data. OpenEvolve has the potential to accelerate the discovery of improved regression algorithms by automating code-level experimentation and reducing dependence on human trial-and-error. We are going to describe what OpenEvolve is, how it works, and why it is relevant to the team's research

3.2 Background: Evolutionary Code Optimization

Traditional algorithm development depends on manual experimentation. Developers have to iteratively adjust parameters, rewrite portions of code, and measure performance until wanted results are reached. This process simply put is slow.

Evolutionary optimization looks to a different angle to the problem inspired by natural selection. In the examples, a population of candidate solutions evolves under selective pressure. Poorly performing algorithms are discarded, and the promising solutions are retained,

modified, and combined to produce new generations. Over many iterations, the population converges toward more effective solutions without requiring explicit design rules.

OpenEvolve applies this principle directly to source code rather than just numeric parameters. Each candidate in the evolutionary population is a block of executable code that can be modified, executed, and evaluated. The system uses LLMs as the means to mutate, capable of generating new code that is syntactically correct and logically sound. These LLMs can reason about structure and mathematical relationships, transforming random search into a guided, semantically aware exploration.

By combining the adaptability of evolutionary algorithms with the contextual reasoning of LLMs, OpenEvolve can have continuous, autonomous experimentation. This capability has relevance to UCTP research, where algorithmic improvements depend on subtle interactions between filtering logic, regression models, and data association functions that are difficult to discover manually.

3.3 Overview of the OpenEvolve Framework

OpenEvolve is an open-source adaptation of DeepMind's AlphaEvolve framework. It automates algorithm discovery by coordinating multiple components, a distributed controller, one or more LLM "islands," evaluation pipelines, and a database of program artifacts. Through these components, OpenEvolve supports thousands of concurrent experiments, allowing large portions of the algorithmic search space to be explored at the same time.

3.3.1 Core Architecture

Each OpenEvolve experiment follows this structured cycle:

1. Generation: LLMs propose new versions of a target code segment, modifying existing logic or generating entirely new constructs.

2. Evaluation: Each candidate program is executed against a predefined test harness that measures objective performance metrics (e.g., accuracy, error, or runtime).

3. Selection: Candidates are ranked according to fitness, and the top performers are retained for further mutation.

4. Reproduction: High-performing code fragments are recombined and reintroduced into the population, forming the next generation.

5. Persistence: All artifacts, code, logs, metrics, and metadata are stored in a version-controlled database, ensuring deterministic reproducibility.

This loop continues until convergence criteria are met or a user-specified iteration limit is reached. The framework's design supports multi-objective optimization, allowing users to balance accuracy, robustness, and computational efficiency simultaneously.

OpenEvolve's architecture includes several key technical components:

- Controller Loop: Orchestrates population management, parallel evaluations, and iteration control.

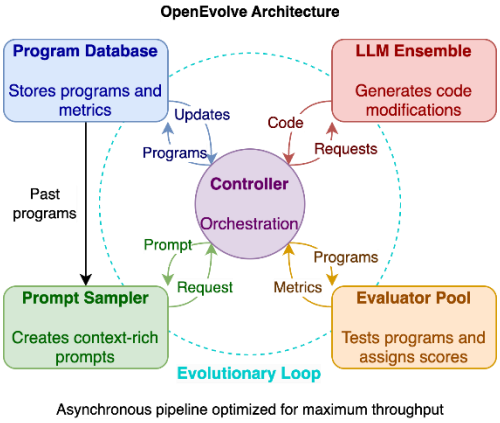
- LLM Ensemble: Multiple language models (e.g., OpenAI, Gemini, Anthropic) operate as independent "islands." They periodically exchange elite solutions to prevent premature convergence and encourage diversity.

- Evaluator: A user-defined Python script that scores each candidate program's output. For this project, evaluators will measure regression accuracy or related tracking metrics relevant to UCTP.

- Database and Artifact Tracker: Captures all source code versions, execution logs, and performance data to guarantee reproducibility across runs.
- Visualization Tools: Command-line and dashboard utilities for monitoring population evolution and comparing versions over time.

Since OpenEvolve integrates with any OpenAI-compatible API, it can run using cloud-based models or local deployments depending on project constraints. OpenEvolve also uses deterministic seeds which allow us to fully reproduce our tests.

Figure 3.3.1.1: OpenEvolve Architecture



What is very important in this architecture as explained above from previous bullet points is that OpenEvolve has a non-blocking (asynchronous pipeline). This means that there is parallelism when computing where the main control orchestration does not wait to finish one task to run another. This is important because it allows the Evolution to generate more quickly. One main problem we have with introducing OpenEvolve's AI architecture into the UCT workflow is that we will need to create a pipeline that corresponds to UCT. This is a big problem because we are separately researching this on our own as a group. Since at the moment UCT is currently still creating their own pipeline to benchmark UCT outputs. So currently the JSON output data we have from UCT is in its raw form. Since the main problem with UCT is that solutions to UCT processing do not have a known solution to compare to. Which is why the usage of OpenEvolve comes in hand since we can help benchmark a lot of the different datasets that are generated by UCT and compare how UCT is performing through OpenEvolve's evaluations.

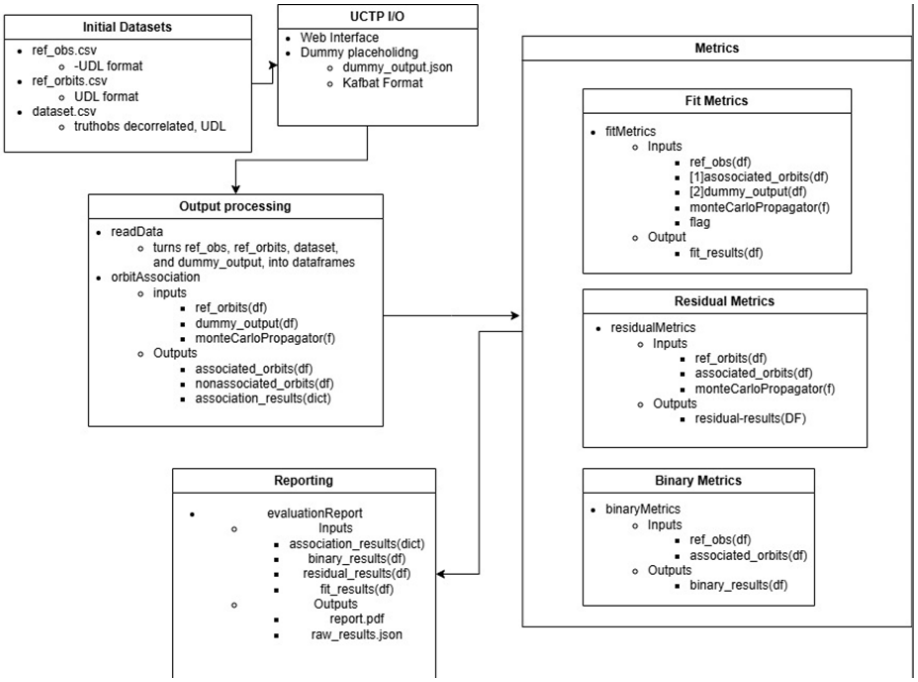
Such datasets to assess include:

- EO Observation Dataset
- TLE Dataset

- UCT Processor Output
- Evaluation Output

When researching UCT processing and OpenEvolve they have many similarities in their pipelines. This includes a configuration file called Config.py and an Evaluation.py file. So, the main research our group has to really dive into is how we can read UCT data into OpenEvolve's architecture. We don't think there are any blockers from the other team when doing research and documentation on UCT. But it is vital that we do research on UCT just like the previous chapter to really understand their architecture to implement it into an AI architecture.

Figure 3.3.1.2: Example of flow architecture of UCT and how they correlate to OpenEvolve metrics



In OpenEvolve our first research was based off a very basic algorithm called function minimization. One of the key things we were looking at is if there is any consistency between different problems that OpenEvolve follows when creating their metric since some of their

metrics consists of LLM generated code. In the function minimization it is shown that there are multiple different scores that OpenEvolve bases their evaluation on. Diversity and complexity is provided by iteration and some of these scores include:

- Value_score: how close we got to the minima in function_minimization. (1.0 best possible result)
- Distance_score: how far our solution is in coordinate space from true optimum solution. (1.0 means exactly optimum)
- Combined_score: Overall fitness combining value and distance. (combined_score = value_score + distance_score)
- Reliability_score: consistency and robustness of model per iteration. (1.0 all inputs tested are stable with no failed states)

When we correlate this back to UCT workflow, we can see how UCT also has their own metrics that they evaluate own. One of the things we are looking towards into is creating our own LLMs to generate these evaluator/metric programs for OpenEvolve. At the moment we are planning on training the model based off of OpenEvolve's problems. But in order to get closer to UCT architecture we might have to train the LLM to understand the knowledge and workflow of how UCT is generating their data. This creates another problem with our group because we would like to make our LLM and GUI public but if we are training the LLM with SDA TAP lab data, then we might not be able to keep it public. We understand as well that our sponsors also want our GUI in order to track OpenEvolve real time as well.

3.4 The Evolutionary Process in OpenEvolve

The operational core of OpenEvolve lies in its evolutionary coding process, which merges principles of natural selection with LLM-driven creativity. The framework replaces traditional random mutation with semantic mutation, which means that the language model proposes targeted edits that preserve code structure and intent while exploring new algorithmic possibilities.

Each OpenEvolve run begins with an initial program, typically a baseline implementation of a known algorithm. The system identifies which sections of code are “evolvable” using explicit tags (e.g., # EVOLVE-BLOCK-START / # EVOLVE-BLOCK-END). These boundaries define safe regions where the model can experiment without breaking the program interface or execution flow.

Once configured, OpenEvolve iteratively generates and tests new code variants. Each candidate is executed within a sandbox environment, and its performance is measured through the evaluator. The evaluator then returns a scalar fitness score, such as a negative mean squared error for regression tasks or accuracy for classification problems. Candidates with higher scores are retained and mutated further, while less successful versions are discarded.

3.4.1 Multi-Objective Optimization

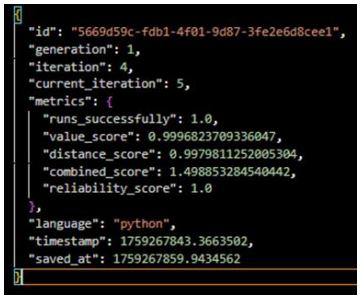
OpenEvolve’s optimization process is guided by the MAP-Elites algorithm, which maintains a diverse population of high-performing programs across multiple dimensions. This approach allows the framework to discover not just the “best” algorithm by a single metric, but a set of elite solutions that vary in trade-offs between accuracy, complexity, and computational cost.

Which for UCTP related problems the flexibility is great. The tracking algorithms have to balance accuracy against runtime constraints and noise resilience. MAP-Elites provides a mechanism to explore those trade-offs automatically, yielding multiple viable algorithmic candidates for further analysis.

3.4.2 Artifact Feedback and Stability

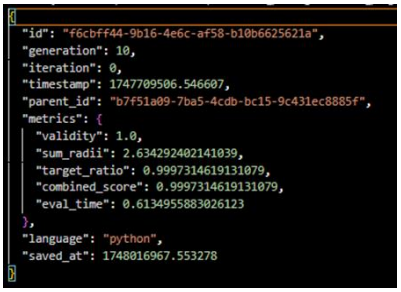
One of OpenEvolve's distinctive features is its artifact feedback channel. During execution, all compile-time and runtime errors are captured and fed back to the LLM, which uses this information to avoid similar failures in the next generations. This feedback loop accelerates convergence by ensuring that the search process focuses on syntactically valid and legible code. Over successive generations, the population stabilizes around more efficient and executable programs

3.4.3 Example Evolution Metrics and Scoring Interpretation



```
{
  "id": "5669d59c-fdb1-4f01-9d87-3fe2e6d8cee1",
  "generation": 1,
  "iteration": 4,
  "current_iteration": 5,
  "metrics": {
    "runs_successfully": 1.0,
    "value_score": 0.9996823709336047,
    "distance_score": 0.997981125200 5304,
    "combined_score": 1.498853284540442,
    "reliability_score": 1.0
  },
  "language": "python",
  "timestamp": 1759267843.3663502,
  "saved_at": 1759267859.9434562
}
```


Figure 3.1



```
{
  "id": "f6cbff44-9b16-4e6c-af58-b10b6625621a",
  "generation": 10,
  "iteration": 0,
  "timestamp": 1747709506.546607,
  "parent_id": "b7f51a09-7ba5-4cdb-bc15-9c431ec8885f",
  "metrics": {
    "validity": 1.0,
    "sum_radii": 2.634292402141039,
    "target_ratio": 0.9997314619131079,
    "combined_score": 0.9997314619131079,
    "eval_time": 0.6134955883026123
  },
  "language": "python",
  "saved_at": 1748016967.553278
}
```


Figure 3.2

OpenEvolve's framework records detailed metrics for every generation and iteration of an experiment. These metrics provide quantitative insight into how the evolutionary process progresses and how each candidate program performs relative to the objective function defined in the evaluator.

Figures 3.1 and 3.2 show two such outputs captured from the initial Function Minimization and Circle Packing experiments. Each figure is a snapshot of one evolved candidate, including unique identifiers, generation count, and performance metrics.

In the Function Minimization example (Figure 3.1), OpenEvolve tracks metrics such as value_score, distance_score, and reliability_score. These correspond to how closely the evolved algorithm's solution approaches the global minimum, how consistent its results are across

repeated runs, and whether the code executes successfully. The high combined_score value (approximately 1.49) points to a strong-performing candidate, reflecting convergence toward optimal behavior in fewer iterations.

In contrast, the Circle Packing example (Figure 3.2) evaluates geometric efficiency. Metrics such as sum_radii, target_ratio, and validity measure how well the evolved algorithm maximizes total circle area while maintaining non-overlapping configurations within the unit square. The combined score of 0.999 demonstrates OpenEvolve's ability to evolve near-optimal spatial solutions.

Together, these outputs demonstrate how OpenEvolve's evaluator framework translates qualitative algorithmic improvements into quantitative performance metrics. Each evaluation result becomes part of the artifact feedback loop, informing the next generation of LLM-driven mutations. Over time, this feedback produces measurable improvements in stability, efficiency, and accuracy, key requirements for transitioning the framework toward UCTP and regression-based research applications.

3.5 Relevance to UCTP and Project Goals

The decision to research OpenEvolve in this project is driven by the want for automated, scalable algorithm discovery for UCTP purposes. UCTP algorithms depend on regression and prediction models that can reliably infer target motion and correlation under uncertain data conditions. Manually refining these algorithms through parameter tuning or incremental code adjustments is inefficient and limited in scope. By defining regression evaluation scripts as part of the framework's "evaluator" function, the team can evolve and assess candidate algorithms continuously, using the same datasets and metrics that underpin existing UCTP experiments.

While symbolic regression will be covered in a separate chapter, it is worth noting that OpenEvolve has already shown strong performance in this area. We will be using symbolic regression benchmarks primarily as testbeds to evaluate OpenEvolve's effectiveness in evolving interpretable, physics-based algorithms. These experiments provide a controlled way to measure how well the framework generalizes to real-world UCTP regression problems.

3.6 Implementation Plan

To assess OpenEvolve's suitability for UCTP-related regression tasks, several validation experiments have been tested prior to integration into the main project. These experiments have helped us understand the tool's behavior, performance limits, and reproducibility under certain conditions. There is also more testing to be done with changing LLM's and their weights in the matter.

The implementation begins by setting up the OpenEvolve environment using the public PyPI release. The framework is configured through a YAML file that defines the language model backend, population size, number of generations, evaluator path, and output directory. Deterministic random seeds are used for each run to ensure that experiments can be exactly reproduced.

Two proof of concept experiments (the Function Minimization and Circle Packing tests) serve as baseline validations. Each includes an initial program, defining the evolvable logic bounded by EVOLVE-BLOCK tags, and an evaluator script, which scores each generated variant using a consistent metric.

- In the Function Minimization test, OpenEvolve evolves a simple random search algorithm into one capable of finding global minima efficiently, producing quantitative improvements in value_score and reliability_score.

- In the Circle Packing test, the framework optimizes geometric placement rules for 26 circles, maximizing the sum of radii while maintaining non-overlapping boundaries.

These experiments verify the core loop of mutation to evaluation to selection, ensuring that OpenEvolve operates correctly before extending it to regression-based UCTP applications. Once verified, new evaluators will be developed to test regression accuracy, stability under noisy input, and runtime efficiency.

3.7 Benefits and Challenges

OpenEvolve provides several advantages over conventional manual experimentation that we have stated earlier:

- Automation: The framework autonomously explores large algorithmic search spaces, reducing human bias and repetitive testing.
- Reproducibility: Every experiment is deterministic, with fixed seeds and full artifact tracking, ensuring consistent and auditable results.
- Scalability: Multiple model “islands” run in parallel, enabling large-scale testing on distributed hardware.
- Continuous Improvement: The artifact feedback channel allows each generation to learn from previous errors, accelerating convergence.

However, there are some challenges or blockers that may show up such as:

- Computational Cost: Each generation requires multiple LLM calls and program executions. Large-scale experiments can incur significant processing and API expenses.
- Interpretability: Evolved code may be syntactically valid but conceptually complex, requiring careful review to ensure physical or logical consistency.

- Integration Complexity: Connecting OpenEvolve's output with UCTP simulations will require interface wrappers and data validation layers.
- LLM Variability: Different model backends (e.g., GPT, Gemini, Claude) can produce subtly different mutation styles, necessitating calibration for consistent results.

3.8 Expected Outcomes

Specifically talking about OpenEvolve by the end of the research portion, we expect to achieve the following outcomes:

1. Validated OpenEvolve Configuration: A stable setup capable of executing hundreds of reproducible evolutionary runs.
2. Demonstrated Algorithmic Evolution: Documented improvements in regression and optimization benchmarks, confirming that OpenEvolve can meaningfully evolve source code logic.
3. Baseline Metrics for UCTP Research: Quantitative benchmarks of how evolved algorithms perform against traditional hand-crafted ones.
4. Scalable Experimentation Pipeline: A portable OpenEvolve configuration that can be reused by other teams investigating algorithmic adaptation in defense or tracking systems.

3.9 Summary and Future Work

Research into OpenEvolve will provide valuable insight into how large language models can be leveraged to drive autonomous algorithm discovery and optimization. By integrating principles of evolutionary computation with LLM-based code mutation, OpenEvolve enables continuous experimentation and guided improvement. Instead of optimizing fixed parameters, OpenEvolve optimizes code itself.

Moving forward, our team's research will shift toward regression-based and UCTP-specific testing. This includes evolving linear and symbolic regression algorithms to assess OpenEvolve's ability to identify interpretable, high-accuracy models. The eventual goal being to embed OpenEvolve into the UCTP research pipeline to evolve components such as data-association cost functions, gating thresholds, and track management logic.

Ultimately, the integration of OpenEvolve within UCTP research supports a broader scientific goal: using AI-assisted evolution to accelerate discovery, improve robustness, and reduce the time required to identify new algorithmic approaches.

Symbolic Regression: Overview & Data Breakdown 5.1

5.1 Introduction

LLM-SR is a framework for data driven scientific equation discovery. It integrates a pre-trained LLM with symbolic regression, evolutionary optimization, and parameter fitting, allowing it to infer underlying physical or mathematical relationships from raw data without prior knowledge of domain specific equations.

5.2 Symbolic Regression - Foundation

Goal: build a baseline equation discovery. A type of machine learning algorithm; predict y given x. Searches for a mathematical equation that best fits a given dataset. Instead of just learning numbers (like linear regression) it tries to discover both the form and parameters of the equation. Symbolic Regression algorithm generates random combinations of math operations, constants, and variables, evaluates fitness where each equation is tested on your dataset. So, the closer its prediction is to the higher its fitness score.

It then takes the crossover where two good equations are selected and the parts of their structure combining genetic material:

Figure 5.21

```
Parent 1: y = x^2 + 3
Parent 2: y = sin(x)
Start with symbolic regression: discovering the analytical form of a
   function
Offspring: y = sin(x) + 3
on f(x) = y
```

Represent equations as binary trees of mathematical operators (+, -, /, *, sin, etc). Apply genetic programming to evolve equations — mutate operators, and crossovers between good

equations, and score each on dataset fitness. Fitness is measured with Mean Squared Error between the predicted and true outputs.

Maintaining interpretability: in machine learning means designing or choosing models that remain understandable to humans, so that we can explain why a model makes a certain prediction, not just what the prediction is.

5.3 Our Expected Result

Baseline SR achieves interpretability but limited exploration efficiency due to random search over a large equation space. LLM-SR Guided Equation Generation

Goal: Replace brute-force symbolic regression search with LLM-based hypothesis generation. By using a pre-trained LLM ($\pi\theta$) we can generate equation program skeletons, such as:

Figure 5.31

```
def f(x, params):
    return params[0]*np.sin(x) + params[1]*x**2
```

F is the set of all the sampled equations. The notation $f \sim \pi\theta$ means each f is drawn from the model. This will continue to generate multiple functions and store into a dataset D = {(xi, yi)}. Then we use:

$$\text{ScoreT}(f, D) = -\text{MSE}(f(xi), yi)$$

Which will keep the highest scoring equation and discard invalid or non-executable ones. Aims to maximize the reward ScoreT (f, D) for a given scientific problem T and dataset D:

$$f* = \arg\max f \, Ed \in D[\text{ScoreT}(f, D)]$$

After each equation is generated, they test how well it fits the data D. The score is a fitness measure (measured with negative mean squared error (-MSE) between the predicted and actual data points). Higher score = better match to real data. Trying to find the equation that gives the best fit to the dataset. It will also use interpretation — so if the prompt mentions" oscillator" it will generate sin/cos/exponential terms rather than random operators, which will drastically reduce the combinatorial search space compared to the symbolic regression approach. For a given scientific problem T and dataset D:

- T = the current problem (discovering the equation for an oscillator)
- D = the data for that problem (input/output pairs)

$$f* = \arg\max f \, Ed \in D[\text{Score}T(f, D)]$$

Finds the equation $f*$ that achieves the best average score across the dataset. Ed means average over all data points in D. arg maxf → the function f that gives the highest score. Overview: The LLM ($\pi\theta$) is repeatedly asked to generate possible equation templates and each generated equation is tested on the real data, scored based on how accurate it is, and the process repeats until it finds the equation f that best matches the data for that scientific problem.

Summary: Navigates through search space more efficiently and discovers more accurate equations with better out-of-domain generalization. Traditional symbolic regression uses search heuristics (genetic algorithms, reinforcement learning, or random sampling). These methods do not know physics, math, or domain logic — they just try operators until something fits the data. LLM-SR changes that by adding an LLM that already understands math structure, physics, and code syntax. It navigates the search space more efficiently. Traditional methods treat equation discovery as blind search — each equation is a random combination of operators and constants, and they evaluate millions of times to find one good fit.

5.4 Hypothesis Generation

Goal: Leveraging a pre-trained LLM to automatically propose diverse and scientifically meaningful equation skeletons by acting as a hypothesis engine. It will propose new mathematical functions def f(x, params): using context, problem descriptions, and evaluation instructions to produce an interpretable orbital model.

What the Prompt Needs:

• Instruction: Directs the LLM to complete the function so that it reflects physical meanings and relationships among input variables. Tells the LLM how to write the equation.

• Problem Specification: Gives context about what problem it's solving, includes key variables, constraints, and objectives. Example: "model an oscillator using position xpos, ypos, zpos, xvel, yvel, zvel."

• Evaluation and Optimization Function: Explains how the generated equations will be judged (e.g., scored based on MSE). This helps it produce structures that are measurable and testable.
• Experience Demonstration: Gives examples of previous good equations, showing what a successful equation looks like and how it improved in past rounds. This lets the LLM learn from proper attempts without retraining.

5.4 Sampling and Validation Process

At each iteration t, the LLM-SR will generate a batch of candidate question skeletons for diverse hypothesis exploration by adjusting temperature:

• Higher temperature → more creative, random outputs.

• Lower temperature → safer, deterministic outputs.

Tune to get a good balance of new ideas and sensible ones. Sampled equation programs are executed, and those failing to execute or exceeding a maximum execution time threshold are discarded.

Once the LLM writes code, it gets tested. If it crashes (syntax error, undefined variable) or takes too long, it's thrown out. Only valid runnable equations move to the next stage where they'll be tested on data.

The LLM's in-context learning and crossover capabilities are employed to refine the suggested equation skeletons iteratively.

By representing equations as Python programs, we take advantage of the LLM's ability to generate structured, executable code while providing a flexible and effective way to represent general mathematical relations.

The program representation also facilitates direct and differentiable parameter optimization to better optimize the coefficients or constants in the generated equations.

5.5 Optimization and Scoring

Goal: To adjust the parameter vector, params, to minimize the prediction error using Mean Squared Error loss function.

- NumPy and BFGS (scipy.optimize): A nonlinear optimization algorithm that uses gradients to find a local minima. It is effective for smooth, continuous functions and problems with relatively few parameters.
- PyTorch + Adam (torch.optim): A stochastic gradient-based optimizer to handle large or noisy datasets.
- BFGS (Broyden Fletcher Goldfarb Shannon) provides accurate deterministic fitting ideal for smaller problems with fewer parameters. Adam combines flexibility and scalability, which is good for noisy or high-dimensional data.

An optimizer uses gradient descent of a loss function to know which direction to move the parameters. “Stochastic” means random — in optimization, it refers to updating parameters using a random subset of the data rather than the entire dataset. The randomness helps make training faster, escape local minima, and introduce exploration in optimization.

After optimizing the skeleton parameters, we assess the fitness of the equation hypothesis by measuring its ability to capture underlying patterns in the data. We compute predicted target values such as:

$$\hat{Y} = f(x, params*)$$

The fitness evaluation score s is then calculated as the negative mean squared error between predicted and true target values. Mean squared error is a loss we want to minimize — lower MSE means a better model, and the higher MSE means worse. But in search-based or reinforcement learning frameworks, we maximize a score (reward), not minimize a loss.

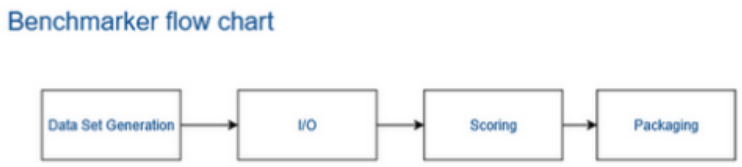
This is defined as:

$$Score = -MSE(\hat{y}, y)$$

This flips the direction of optimization — if MSE is small (good), s becomes a large negative number (closer to zero, higher). If MSE is large (bad), s becomes a more negative number (lower).

5.6 Data Generation

Figure 5.61



Test datasets are created based on the equation-based output (e.g., predicted orbital positions, velocities, or observed data) formatted according to the Unified Data Library schema to ensure compatibility with the UCTP processor.

- The I/O: This is generated and transferred into the UCTP benchmark system through standardized I/O operations. This is where we will store the JSON structure that contains observational and reference information to be scored accurately.
- Scoring: Compare with the truth dataset and compute performance metrics to quantify how well the LLM-SR generated equation explains or predicts observed data.
- Packaging: Export as a JSON and summarize into a report for detailed analysis. This validates the LLM-SR's discovered equations against the UCTP standardized benchmarking framework.

5.7 Testing and Evaluation

Figure 5.71

```
root@docker-desktop:/app# cd /app/examples/symbolic_regression
python problems/phys_osc/PO37/evaluator.py problems/phys_osc/PO37/initial_program.py
Evaluating model: problems/phys_osc/PO37/initial_program.py
Using NUM_INPUT_FEATURES_EXPECTED = 3
Using MODEL_NUM_PARAMS_EXPECTED = 10
Loading X_train from: ./problems/phys_osc/PO37/X_train_for_eval.npy
Loading y_train from: ./problems/phys_osc/PO37/y_train_for_eval.npy

Evaluation Results:
  combined_score: 1.6927
  negative_mse: -0.0203
```

Each team member completed evaluation logs from running the symbolic regression benchmark for the physical oscillator problem. This evaluation script tests the generated symbolic equation within initial program.py against the ground-truth dataset (X train for

eval.npy and y train for eval.npy) to assess how well the discovered function models the target dynamics. In terms of what the model expected:

- 3 input variables
- 10 tunable parameters to be optimized during evaluation
- Loading input and output training data

Evaluation Results:

- Combined score: 1.6927 → this is the overall metric combining accuracy and interpretability, where higher values represent better performance.
- Negative mse: -0.0203 → this number represents the negative MSE between predicted and true outputs; since it's closer to zero, it indicates higher accuracy.

Overview: Since we generated predictions with a low error rate, we can conclude that the model successfully suggests the discovered equation is both executable and reasonably accurate for this oscillator dataset. The evaluation module has correctly loaded, validated, and scored the generated equation structure using the defined scoring.

Figure 5.72

Dataset	Time range	initial values	F	α	β	δ	γ	ω
Oscillator 1	(0, 50)	{x=0.5, v=0.5}	0.8	0.5	0.2	-	0.5	1.0
Oscillator 2	(0, 50)	{x=0.5, v=0.5}	0.3	0.5	1.0	5.0	0.5	1.0

Table 2: Parameter values for Oscillator datasets.

Oscillator 1:

$$\dot{v} = F\sin(\omega x) - \alpha v^3 - \beta x^3 - \gamma x \cdot v - x\cos(x)$$

Oscillator 2:

$$\dot{v} = F\sin(\omega t) - \alpha v^3 - \beta x \cdot v - \delta x \cdot \exp(\gamma x)$$

5.8 Nonlinear Oscillator Equations

Nonlinear Oscillators are chosen to test whether the model can discover true physical relationships when evaluating and not just memorizing equations. Nonlinear equations can also involve non-convex optimization challenges and chaotic behaviors forcing the LLM-SR to reason symbolically about the equation under the hood. This is an important benchmark stress test, ensuring the algorithm can consistently discover accurate and interpretable equations, even when faced with highly complex or unstable dynamics.

Summary Moving forward we plan to discuss how each stage of the LLM-SR pipeline — OpenEvolve–UCTP benchmarking — fit together. The goal will be to understand how our current approach can be extended toward more complex applications with the information given we have. Therefore, by aligning our implementation with more advanced datasets and mathematical representations, we can improve the model's ability to handle high-dimensional datasets.

Graphical User-Interface Development:

6.1 Introduction

The OpenEvolve Dashboard represents an intersection of artificial intelligence, web technologies, and systems evaluation. Built with the MERN stack MongoDB, Express.js, React, and Node.js this platform serves as a dynamic web-based interface for analyzing the output data of OpenEvolve's program evaluation framework. Designed to be extensible and modular, the system provides users with the ability to visualize program metrics, interpret AI-driven evaluations, and interact directly with backing evaluation scripts in an integrated, browser-accessible interface.

The dashboard is deployed as a single-page application (SPA), which provides fast, smooth transitions among analysis views without unnecessary server reloads. The user can dive into in-depth data metrics, observe visual analytics, and call upon LLM-based reasoning to explain output results, creating a unified and interactive analytical.

6.2 MERN Stack

The decision to utilize the MERN stack is consistent with our emphasis on scalability, speed, and cross-platform compatibility. Each component plays a specific and complementary role:

- MongoDB: a NoSQL database that supports JSON-like document structures, ideal for flexible data storage such as metric results, evaluation logs, and user preferences. Although OpenEvolve can operate without persistent storage, MongoDB remains an optional enhancement to log metrics across sessions and enable comparative analysis between iterations.

- Express.js: the backend web application framework that connects Node.js with React. It facilitates secure API routes that retrieve processed data from the OpenEvolve evaluator or other server-side scripts.
- React: the foundation for the front-end interface, providing the SPA experience. React components organize data visualization panels, user controls, and dynamically update metric outputs through hooks and context states.
- Node.js: the runtime environment that executes server-side JavaScript, hosting both the Express backend and the interaction layer with OpenEvolve's Python-based modules.

Together, these technologies create a robust ecosystem where the user interface, server logic, and AI evaluation modules communicate in real time.

6.3 Database Considerations

While MongoDB is fully integrated into the stack, its necessity is conditional. OpenEvolve's computational workflow, particularly its Python-based analysis scripts generates data that can be directly rendered on the front end without long-term storage. In scenarios where persistence is not required (e.g., single-run evaluation), JSON-based memory storage or temporary API state management suffices.

However, for research reproducibility and user-specific metrics tracking, MongoDB provides clear advantages:

- **Historical Comparison:** Stores past OpenEvolve runs for cross-metric analysis.
- **User Personalization:** Saves display settings and visualization filters.
- **Team Collaboration:** Enables shared access to previous evaluation results through user-specific collections.

This modular approach to database use allows the project to remain lightweight while still supporting scalability for larger research or production contexts.

6.4 Front-End Visualization and Metrics Dashboard

The OpenEvolve Dashboard emphasizes clarity, accessibility, and precision in visualizing computational data. Its metrics page central to the user experience aggregates the output data from OpenEvolve's metrics and presents it in visual forms. These data visualizations allow users to interpret performance, algorithmic efficiency, and other key measures in real time.

Each data update triggers a React state refresh, automatically redrawing the metrics display without requiring manual reloads. The goal is to present technical evaluation data in a format that is both intuitively readable and scientifically rigorous.

Figure 6.41: Inspiration GUI

Figure 6.42: Current Developed GUI

6.5 Integration Between OpenEvolve and Web Platform

The integration between OpenEvolve and the dashboard will be established through a secure API bridge. Node.js executes subprocess calls to Python, invoking evaluator.py and init_program.py. Once computation completes, the results are formatted into JSON and sent to the front end via Express routes.

React then consumes this data and displays it through its visualization components. This design enables high interoperability between the AI system's Python backend and the JavaScript-driven web client, creating a unified workflow for research analysis.

6.6 OpenEvolve Architecture Versus Traditional Stacks

OpenEvolve already has a graphical user interface to showcase how the algorithms are evolving and so we are following that architecture and the tech stack to use so. The difference here is that instead of creating a web application where we need to host a server for a database. Everything will be held within the directory of the files being created and manipulated from each OpenEvolve iteration. The current tech stack that was identified from going over the visualizer python file embedded in OpenEvolve files are:

OpenEvolve Visualizer.py tech stack:

- JSON data creation and retrieval
- Python + Flask
- HTML/CSS/JSS

Environment:

- Real-time API scans per checkpoint
- All local based data not using a server since the High-Performance Computing cluster itself will be storing everything

Example imports from Visualizer.py:

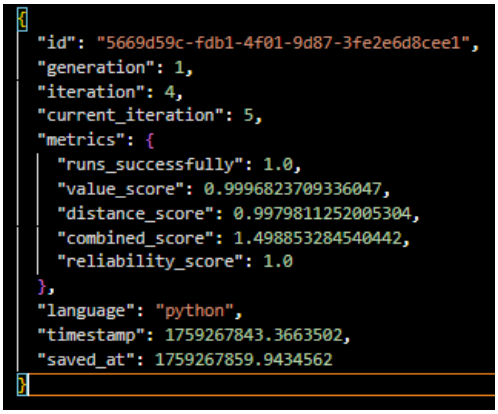
Figure 6.61 Imports that OpenEvolve uses

```
import os
import json
import glob
import logging
import shutil
import re as _re
from flask import Flask, render_template, render_template_string, jsonify
```

The main problem of OpenEvolve's architecture is the idea that we can't have a consistency of how the metrics are per algorithm. What this means is that each different problem in OpenEvolve has their own schemas on the metrics that they use to evaluate how efficient OpenEvolve is doing. An example of this is that our sponsor wants a real-time line graph to show how OpenEvolve is doing so they can check up on the status of it and also identify when to stop the iterations. Since in some cases OpenEvolve finds the optimized code and solution fairly quickly even though you set the iteration bound to 100.

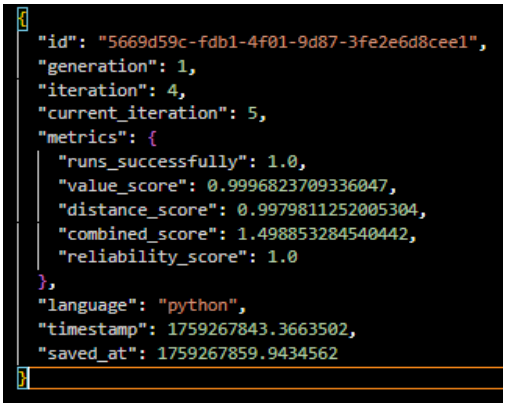
The next two figures will show how the different algorithms have different metrics. Talking to the sponsor, they wanted to be able to use this GUI in the future for other problems as well and one thing that was very beneficial from research that we gathered is the fact that we can make our dashboard dynamic. What we plan to do is we will loop through the metrics to find what is being evaluated to whatever algorithm we are using and then create the dashboard graphs from that. And so, our website will have dynamic sizing based off how many metrics are being used. The user will also have the ability to filter certain graphs(metrics) to their liking if they want to just focus on certain portions of the dashboard.

Example metrics from OpenEvolve's code:



```
{
  "id": "5669d59c-fdb1-4f01-9d87-3fe2e6d8cee1",
  "generation": 1,
  "iteration": 4,
  "current_iteration": 5,
  "metrics": {
    "runs_successfully": 1.0,
    "value_score": 0.9996823709336047,
    "distance_score": 0.9979811252005304,
    "combined_score": 1.498853284540442,
    "reliability_score": 1.0
  },
  "language": "python",
  "timestamp": 1759267843.3663502,
  "saved_at": 1759267859.9434562
}
```


Figure 6.61 Function_minimization



```
{
  "id": "5669d59c-fdb1-4f01-9d87-3fe2e6d8cee1",
  "generation": 1,
  "iteration": 4,
  "current_iteration": 5,
  "metrics": {
    "runs_successfully": 1.0,
    "value_score": 0.9996823709336047,
    "distance_score": 0.9979811252005304,
    "combined_score": 1.498853284540442,
    "reliability_score": 1.0
  },
  "language": "python",
  "timestamp": 1759267843.3663502,
  "saved_at": 1759267859.9434562
}
```


Figure 6.62: Circle_Packing

So how these are specified are:

Diversity and **Complexity** is provided per iteration, so we will just need to create a time series line chart to show the efficiency changes per iterations.

Some cases to look at are the **feature_stats** and the **combined_score** parametrics provided in the **metadata.json**.

- **value_score:** how close we got to the minima in function_minimization. (1.0 best possible result)
- **distance_score:** how far solution is in coordinate space from true optimum. (1.0 exactly optimum)
- **combined_score:** overall fitness combining value and distance. (combined_score=value_score+distance_score)
- **reliability_score:** consistency and robustness (1.0 all inputs tested are stable no failed states)

In the Metadata json file, this is how we will showcase in the dashboard of the dynamically created graphs based off each metric. The overall performance graph will be combined_score.

6.7 OpenOnDemand

OpenOnDemand is a way where we can access MITRE supercomputers through the web. One of the big problems we needed to take care of is the idea of consistency between all the software being used. This includes Python versions and file differences between all the environments. How we combat this is in our GitHub environment, we have a centralized directory to develop the GUI and LLM. The reason why we did this is because as researchers we all have separate directories with our own installation and environments of OpenEvolve where we manipulate and document on.

One of our big blockers is the access to MITRE's HPC. Overall, as a team we can continue to develop the GUI through our own environments, but we can't make sure it works on their HPC since they have specific licenses they are only allowed to use for the company. One of the key things is Dr. Cline (Senior Researcher at MITRE & SDA TAP lab) and the team is working on is working with Docker. We are looking into this because we can isolate the environment to the specific software the HPC allows, and we can test our code there to make the transition from our GitHub to the HPC is seamless. Just the same way we can run example problems through our command prompts through local machines, we can do the same with building and running through docker.

Large Language Model Development:

7.1 Overview

The integration of a large-language model (LLM) into the OpenEvolve Dashboard enhances the platform by offering the ability to produce evaluator.py and init_program.py. Natural-language explanations and contextual reasoning about the system's metrics could also be offered to enhance its use. Specifically, the dashboard leverages ChatGPT, which serves as an on-demand research assistant. Rather than functioning as an open-ended chatbot, the model is strictly scoped to interpret and assist with two core modules of the system: evaluator.py and init_program.py.

7.2 Architecture of LLM Integration

The architecture for LLM integration resides within the MERN-stack framework defined in the Tech Stack chapter, linking user interactions through React (frontend) to Node.js/Express (backend), which in turn dispatches communication to the ChatGPT API. The workflow proceeds as follows:

1. User Input – The user submits a query via the "AI Assistant" panel on the dashboard.

2. API Endpoint Handling – An Express route receives the query and constructs a prompt for the ChatGPT API.

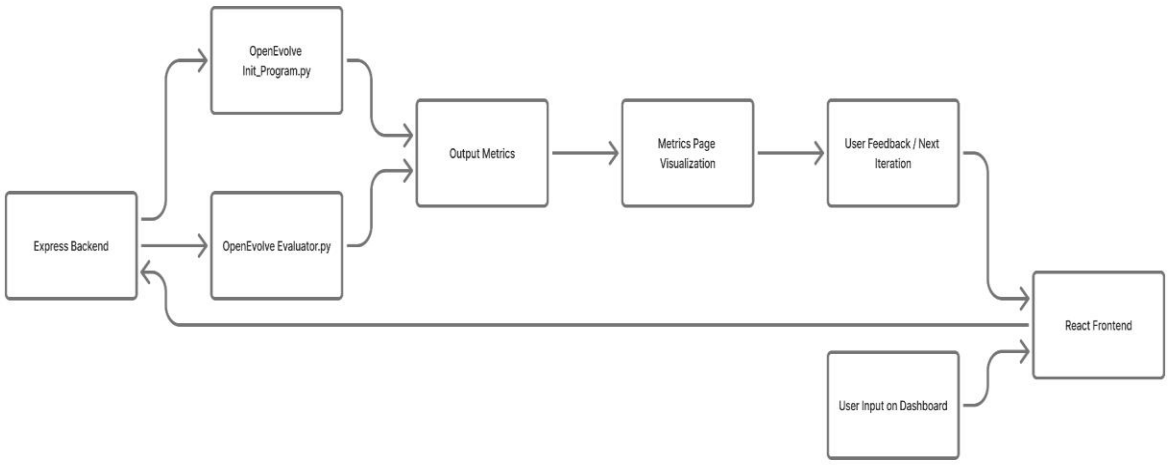
3. Prompt Construction – The backend composes a context string that includes relevant code fragments followed by the user's question, thus bounding the model's scope to only the domain-relevant content.

4. Model Invocation – The prompt is sent to the ChatGPT API and the model returns a response with the now start for evaluator.py and init_program.py.

5. Response Filtering and Sanitization – The backend examines the model's answer to ensure compliance with the domain constraints, eliminating any off-topic or irrelevant content.

6. Frontend Delivery – The sanitized explanation is returned to the React frontend and displayed in the "AI Assistant" panel for the user's review.

Figure 7.21: Developed schematic for GUI and LLM



7.3 Data Handling, Privacy, and Context Management

Because OpenEvolve could deal with potentially sensitive data, the integration design prioritizes privacy, transparency, and minimal data exposure. Key safeguards include:

- Only non-sensitive code excerpts from evaluator.py and init_program.py are forwarded to the model. Full datasets, user-specific metrics, or raw logs are never sent to ChatGPT.

- The model's context window is strictly limited: only the relevant modules' code plus the user query, thereby reducing the risk of off-topic drift beyond the intended domain.

- No persistent storage of the model's responses is required unless specifically enabled for logging or auditing.

- The system maintains an audit trail of which code fragments were sent to the model and which user queries triggered them, ensuring reproducibility and transparency of the AI-assisted reasoning.

These practices align with recognized standards for responsible AI usage, ensuring that the LLM functions as a bounded assistant rather than an unfettered general-purpose tool.

7.4 Model Behavior, Scope, and Prompt Engineering

To ensure that the LLM remains focused and effective, we define specific behavioral roles and prompt engineering strategies:

- Role Definition – The system instructs the model explicitly: "You are an assistant for the OpenEvolve Dashboard. You will only interpret the following code fragments and answer questions about how they operate."

- Content Focus – The model is restricted to tasks such as:

    - Explaining how a given function may be implemented and used within OpenEvolve

    - Describing how init_program.py sets up parameters, handles configurations, and orchestrates program execution.

    - Create the evaluator.py file and init_program.py file within the context of the problem

- Prompt Structure – The backend constructs the prompt with the following components:

    - System message specifying the assistant's domain.

- User Input: "I want to find y = 3x + 5"

System sends to model:

"You are creating an evaluator and initial_program for a linear regression problem with formula y = 3x + 5.

Only output code relevant to this symbolic regression task."

- Limitation Handling – Because the free-tier model has finite context size and rate limits:
  - The system may truncate overly long modules and provide links or references instead.
  - If the model's output appears unsatisfactory, the system can retry with an adjusted prompt or notify the user that deeper analysis may require manual review.

7.5 Benefits and Limitations

Benefits include:

- The ChatGPT free model offers a cost-effective way to add interpretive AI to the dashboard without requiring custom-trained models or large infrastructure.
- By focusing strictly on init_program.py and evaluator.py, the assistant helps users understand the internal logic of OpenEvolve's evaluation pipeline, thus enhancing research transparency and user engagement.
- The integration encourages interactive learning—users to ask targeted questions and receive immediate responses, supporting both novice and advanced users of the dashboard.

Limitations we may run into:

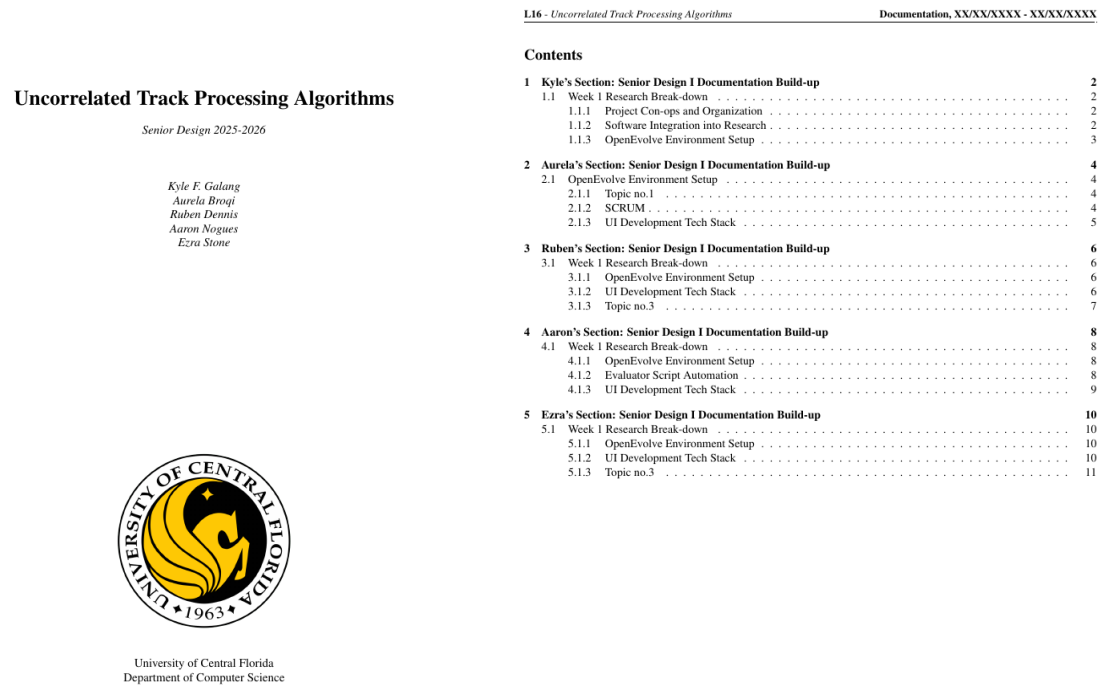
- The free-tier model is subject to rate limits, variable latency, and context-size constraints.
- The assistant cannot inspect live runtime data beyond what is provided in the prompt; it cannot modify behaviour or access private datasets.
- Because only code excerpts are included, the assistant may lack full context of the system's state or external dependencies, which can lead to incomplete or approximate answers.
- Over-reliance on the assistant may lead users to accept explanations uncritically; therefore, manual validation remains essential.

GitHub Environment and Documentation:

8.1 Documentation Overview

Currently how we have our documentation set up is through GitHub and slides. In our GitHub we have a LaTex template that we created to keep documentation consistent and professional and majority of our findings will be done through that. As well as in Jupyter Notebooks. Our slides are also uploaded into GitHub which showcases everything that we show to our weekly sponsor meetings which will also be a digital footprint for our progress throughout the timelines.

Figure 8.11: LaTex Template showcase



Uncorrelated Track Processing Algorithms

Senior Design 2025-2026

Kyle F. Galang
Aurela Broqi
Ruben Dennis
Aaron Nogues
Ezra Stone

University of Central Florida
Department of Computer Science

L16 - *Uncorrelated Track Processing Algorithms* **Documentation, XX/XX/XXXX - XX/XX/XXXX**

Contents

8.2 Research Overview

For OpenEvolve each team-member has installed it into their local machines, in order to keep track of progress and file manipulations. We've all uploaded our environments to GitHub

so that we have file version control in case anything breaks. OpenEvolve's architecture is pretty complex so it is simply changed to the evaluator and initial python files can break how the iterations and metrics are defined. We are also creating our own subset of problems that more closely resemble UCT. Where our primary manipulation in files is having our own symbolic regression files that we change and research on in order to account for more UCT related problems.

Figure 8.21: Different directories for environments

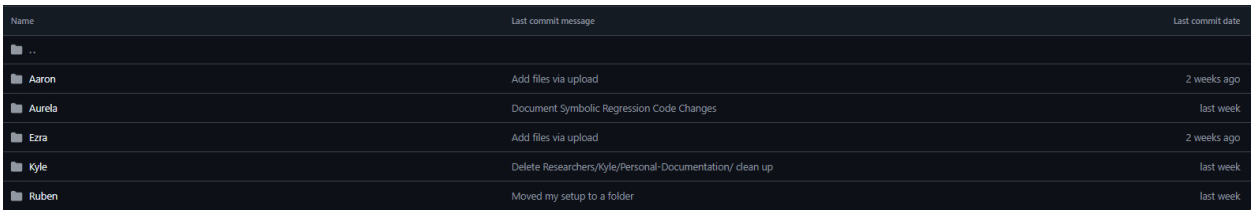


Figure 8.22: OpenEvolve environment showcase

Of course, because we are uploading our OpenEvolve environments into GitHub, we also set up secrets which are basically environment variables for our OpenAI API keys.

8.3 GUI and LLM Development Overview

In our GitHub, we have a centralized directory where we have a single OpenEvolve environment that is constantly being changed by the developers. We did this because in our primary research directories, we are constantly changing variables and files in OpenEvolve and

so we created a directory called single truth which has a clean environment of OpenEvolve that is not touched. What we aim to do with our GUI and LLM development is to be able to contribute to the OpenEvolve GitHub itself where the GUI does not change anything to the existing code but just an extra feature that can be called upon when using OpenEvolve.

We've talked to our sponsors, and they said all research being done is public which means we can contribute to the GitHub repository without needing to hide any of our work. Our research as well can be public and showcased, but the only things that cannot be shared are the UCT data and architecture that we work with. So preliminary research to see the limitations and strengths of OpenEvolve benchmarking symbolic regression problems are public. Our GitHub and Jira are also shared with our sponsors so they can keep track of our documentation, research, and taskings all the to the end of March.